METEOR: From Raw Surround-View Recordings to a Twelve-Task Driving Network with *Zero Human Annotation* and *Zero Human-Written Code*

Dan Umeda
TIER IV, Inc.
https://github.com/tier4/METEOR

Abstract

We present METEOR (Multi-task Estimation of Traffic Elements, Objects and Roads), a surround-view, camera-only network that predicts twelve driving tasks in a single forward pass—BEV lane segmentation, metric depth, oriented 3D boxes, 21-class 2D segmentation, 10-class 2D detection, 3D semantic occupancy with ground-plane flow, multimodal end-to-end driving (three trajectory hypotheses with confidences, steering, acceleration, braking), one-shot multi-agent forecasting with an explicit parked/stopped flag, ego-relevant traffic-light state, a continuous near-range risk field, and a vector lane graph—and, equally importantly, the fully automated pipeline that builds it. **No human annotated a single box, mask, waypoint or light state, and no human wrote a single line of code**: every supervision signal is distilled offline from raw eight-camera recordings of the Co-MLOps Data Recording System by a seventeen-stage autolabel factory built on the CoMET autolabeling foundation, and the complete system—the factory, the network, the trainer, the deployment stack, and this manuscript—was authored by an LLM agent (Claude Fable 5) operating autonomously under human direction. The network couples a shared image backbone with a *depth-gated inverse perspective mapping*, a streaming temporal memory (three ego-motion-warped BEV slots covering 2.8 s), and task routing that serves geometry heads from the single-frame BEV and motion heads from the fused BEV; it restricts itself to TensorRT-exportable operators (45.9 M parameters) and runs all tasks in a single fp16 engine at $\sim$70 ms per eight-camera frame on one L40S. LiDAR, when present, is an *optional* input on the very same weights: projected returns sharpen the predicted depth distribution elementwise, and feeding zeros is bit-identical to the camera-only network, so one engine serves both sensor configurations. A rolling-round training methodology interleaves data conversion, ground-truth refinement and training; we document its measurement-driven corrections—curvature-weighted imitation, camera-confirmation relaxation for vulnerable road users, near-range-first detection supervision, normalisation-statistics isolation for the temporal memory, ε-winner-takes-all against multimodal collapse, an explicit motion residual that halts oncoming-heading flips, and automatic exclusion of GNSS-dead indoor scenes—each of which repaired a quantified failure mode. On a held-out recording day, METEOR reaches 0.302 BEV lane mIoU, 0.535 2D-segmentation mIoU, 3 s trajectory ADE of 0.73 m (0.72 m on curves, down from 2.31 m), near-corridor vehicle recall of 0.72 with 5.8° yaw error, and 0.86 traffic-light accuracy—*all without a single human label*—and the unmodified engine transfers zero-shot to right-hand-traffic US recordings.

1 Introduction

Perception stacks for automated driving are conventionally built one task at a time, each with its own annotation campaign. Labeling is the dominant cost: boxes, masks, lane geometry, and occupancy grids each require dedicated tooling and thousands of hours of human effort, and every new task restarts the cycle. At the same time, modern data-collection platforms record far more information than any annotation budget can absorb—synchronized surround cameras, LiDAR, and centimeter-level ego-motion—which in principle contain almost everything the labels would say.

This paper describes an attempt to close that loop completely. Starting from raw recordings of the Co-MLOps Data Recording System (DRS)[1]—eight synchronized cameras, a concatenated LiDAR, ego-poses, and machine-generated 2D panoptic labels—we build *twelve* supervision signals and train a single multi-task network, with two hard constraints that we believe make the result practically interesting:

- **Zero human labels.** All ground truth—including traffic-light states, per-agent futures, risk fields and the vector lane graph—is produced by an autolabel factory built on CoMET, the autolabeling foundation of the Co-MLOps project. *Humans never annotate; they only record and drive.*
- **Zero human-written code.** The complete system—ground-truth extractors, model, trainer, demo tooling, documentation, and this paper—was written by an LLM agent (Claude Fable 5) working autonomously under human direction (Sec. 8).

Our contributions are:

1. **An autolabel factory for twelve tasks** (Sec. 3): a fourteen-stage, per-scene, resumable pipeline that turns

[1] https://co-mlops.tier4.jp/

raw DRS recordings into BEV lane maps, dense metric depth, camera-confirmed oriented 3D boxes, 21-class 2D segmentation, 10-class 2D boxes, end-to-end driving targets derived purely from ego-motion, 3D semantic occupancy with ray-carved free space, per-agent 3 s futures with stationary flags, ego-relevant traffic-light states distilled from TLR autolabels, a potential-field risk map, and a vector lane graph with adjacency—plus a four-layer cleansing stack (scene, frame, label, loss) that makes such labels trainable.

2. **METEOR** (Sec. 4): a 45.8 M-parameter surround-view network whose depth-gated IPM converts image features to BEV using only TensorRT-friendly operators, extended with a streaming three-slot temporal memory, task routing (geometry heads on the raw single-frame BEV, motion heads on the fused BEV), and winner-takes-all multimodal trajectories ($K=3$); the head design follows an explicit capacity rule—*parameters at low resolution, compute at high resolution*—that adds tasks at $< 5\%$ marginal cost, and the whole graph deploys as one fp16 TensorRT engine at $\sim$70 ms per frame.
3. **A rolling-round training methodology** (Sec. 5): data conversion, GT re-extraction and training run concurrently; each $\sim$24 h round folds in newly converted scenes and measurement-driven loss fixes. We document three such fixes (curvature weighting for imitation, coverage-based thin-class rasterisation, small-object retention) with the quantified failures that motivated them.
4. **An agent-authored system** (Sec. 8): to our knowledge the first published driving-perception system in which not only the labels but all engineering artefacts were machine-generated, with humans reduced to specification and review.

2 Related Work

Backbones and dense prediction. Our encoder follows the standard lineage of convolutional backbones [1–4] with batch normalisation [9] and multi-scale fusion by FPN [8]; transformer encoders [5, 6] and high-resolution networks [7] are attractive but conflict with our strict deployment constraint. Dense prediction heads descend from FCN [10] and its successors [11–14], with efficiency-oriented designs [15, 16] closest to our encoder–decoder segmentation head. Panoptic labels [17, 18] in COCO RLE format [19] are the raw material of our factory, and the class systems echo urban-scene benchmarks [20, 21].

2D object detection. Two-stage [22, 23] and one-stage [24–29] detectors are both well explored; keypoint-based formulations [30, 31] and set prediction [32, 33] removed anchors and NMS respectively. Our 2D head is a three-scale CenterNet [31]: YOLO-style multi-scale assignment [26] with heatmap decoding that reduces to a max-pool, which survives TensorRT export.

Lane and map perception. Image-space lane detectors evolved from spatial message passing [34] through row-anchor and line-proposal methods [35–38] and polynomial regression [39]; 3D lane benchmarks [40–42] moved the task to metric space. Online HD-map construction rasterises or vectorises BEV lanes [43–48]; our factory produces exactly such vectorised lane GT—but offline, from LiDAR accumulation rather than human annotation, in the spirit of classic skeleton-and-simplify cartography [143, 144].

Camera-to-BEV lifting and 3D detection. Splatting a predicted depth distribution [49] underlies a large family of camera-BEV detectors [50–53]; query- and attention-based alternatives include DETR3D [54], PETR [55, 56], BEVFormer [57], sparse recurrent variants [58, 59], and cross-view transformers [60]; simpler bilinear-sampling baselines are surprisingly strong [61–63]. Monocular baselines [64, 65] established camera-only 3D detection. Our depth-gated IPM is a pull-based hybrid: like classic IPM [66, 67] it samples at geometric ground intersections (including fisheye models [68]), but the sample is gated by the predicted depth probability at the ray range—LSS run backwards, with `gather` instead of scatter. LiDAR detectors [69–74] and fusion methods [75, 76] inform our CenterPoint-style BEV head, which we supervise with camera-confirmed LiDAR annotations only.

Occupancy. Semantic scene completion began indoors [77] and on LiDAR [78]; camera-based 3D occupancy is now a benchmark task [79–86]. Our contribution is on the label side: multi-sweep labeled-LiDAR voxelisation with dynamic-object de-smearing and dense ray-carved free space, produced without any human input.

Depth estimation. Supervised monocular depth spans direct regression [87], ordinal and bin formulations [88–90], self-supervised photometric training [91], and large-scale generalist models [92, 93]. Our depth GT densification is closer to classical completion [94, 95], but segment-aware: interpolation runs inside merged panoptic road segments.

End-to-end driving. From ALVINN [96] and PilotNet [97] through conditional imitation [98, 99], driving-by-planning hybrids [100, 101], simulator-centric agents [102–107], and BEV-planning stacks [108–110], imitation from ego-motion is a long-standing recipe. Our E2E head is deliberately minimal (speed-conditioned regression on the shared BEV feature); its interest lies in the free supervision and in the documented imbalance corrections, evaluated with curve-conditioned metrics on real recordings [112, 113].

Datasets and auto-labeling. Public driving corpora [111, 112, 114–117] standardised the schema our recordings follow. Offboard auto-labeling [118, 119] distills strong detectors into labels; the broader semi-/self-supervised toolbox [120–126] motivates training on machine-generated targets. Our factory is detector-free for five of seven tasks and covers modalities (lanes, depth, occupancy, driving) that offboard detection pipelines do not.

Multi-task learning. Sharing one trunk across tasks is classical [127–129]; loss balancing has been addressed by uncertainty weighting [130], gradient normalisation [131], and gradient surgery [132]. We use fixed hand-tuned weights corrected by measurement (Sec. 5.2); with seven tasks and rolling data we found interpretable static weights easier to operate than adaptive schemes.

LLM agents for engineering. Code generation has pro-

gressed from benchmark-level synthesis [145, 146] to tool-using agents [147] evaluated on real repository tasks [148] with rapidly improving frontier models [149]. This project is, to our knowledge, an early data point on a full ML *system* (not a patch or a script) built end-to-end by such an agent, including its own measurement loop (Sec. 8).

3 The Autolabel Factory

The factory consumes t4dataset-format recordings (a nuScenes-like schema [112]: keyframed samples, calibrated sensors, ego-poses, LiDAR sweeps, and 2D panoptic annotations stored as COCO run-length masks). A single driver executes ten stages per scene; every stage skips existing outputs, so the factory is resumable and runs scene-parallel alongside training. We summarise each supervision signal; exact recipes are in the repository documentation.

3.1 BEV lane ground truth

LiDAR points from every keyframe are projected into all cameras (pinhole and equidistant fisheye [68]) and take road-surface classes from the panoptic masks; thin classes (lane line, stop line, road edge, marking) are only accepted within 15 m, where the pixel-to-ground error is sub-cell. Ego-poses accumulate the labeled points into a global 0.1 m grid over the whole scene, which is rasterised with hit-count priorities and a 20 m observation-distance mask. The raster is then *vectorised*—skeletonisation [144], spur pruning, Douglas-Peucker simplification [143], and dash-gap linking—and per-frame GT crops ($\pm 80 \times \pm 50$ m at 0.2 m) are re-rendered hybrid-style: area classes from the raster, line classes redrawn from the vectors at fixed width. Two guards proved essential: enclosed road holes ($\leq$400 cells, LiDAR shadows) are filled, and opposing carriageways are removed by an ego-connected-component filter with a 30% revert guard that protects wide intersections from being wiped.

3.2 Dense metric depth

Per camera, LiDAR points splat a sparse min-depth image at stride 4. Depth is densified—in the spirit of classical completion [94, 95]—by linear interpolation *over the union of road and road-paint panoptic segments*—interpolating paint separately leaves holes around every lane line—followed by a geometric ground-plane fill restricted to rays within 50° of downward (a guard against fisheye-calibration tails), a same-segment nearest fill, and per-segment medians. Sky and ego pixels receive fixed codes. The result is a 64-bin (1.25 m) target at 108×192 for all eight cameras.

3.3 Camera-confirmed 3D boxes

The recordings carry LiDAR-frame 3D annotations whose instance IDs are disjoint from the 2D panoptic instances, so association must be geometric: a 3D box is kept only if its eight projected corners overlap a same-class 2D annotation in some camera (intersection over minimum area > 0.3). This removes fully occluded objects that a camera-only network cannot see, which otherwise poison the BEV detection loss.

3.4 2D segmentation and 2D boxes

Both taxonomies are CSV-driven so that class definitions are data, not code: 21 semantic classes and 10 instance classes with Cityscapes-like colours. Two findings matter. (i) *Thin classes do not survive naive downsampling*: at stride-4 GT resolution a lane line is $\sim$0.3 px wide and nearest-neighbour resizing deletes it; rasterising a cell as the thin class when $> 12\%$ of its footprint is covered recovered $+49\%$ lane pixels and turned broken dashes into connected supervision. (ii) *Silent truncation deletes exactly the rare classes*: with a per-camera box budget of 32, area-sorted retention dropped 29% of annotations in crowded scenes—all of them small (cones, traffic lights, distant unknowns). Raising the budget to 96 and adding per-class positive weights fixed the small-object classes.

3.5 End-to-end driving targets from ego-motion

No CAN bus data is available, so all driving targets derive from ego-poses: future positions at $+0.5 \ldots +3.0$ s transformed into the current ego frame (six waypoints); speed and acceleration by smoothed finite differences; steering via a bicycle model $\delta = \arctan(L\dot{\psi}/v)$ with wheelbase L=2.8 m, masked below 0.5 m/s; braking as acceleration below -0.5 m/s^2. Instantaneous signals are written for *every* frame—an early version left the speed field zero when the 3 s future was missing at scene tails, and the deployed model, conditioned on that bogus zero, predicted "hold" at 80 km/h. Trajectory targets carry a validity mask instead.

3.6 3D semantic occupancy

Ego-frame voxels ($16 \times 200 \times 200$ at 0.4 m; ± 40 m, $z \in [-1, 5.4)$ m) with ten classes plus *unknown*. Points from ± 8 strided frames are labeled by sampling the cached 21-class segmentation GT (a factor-15 speedup over re-decoding panoptic masks) and accumulated via ego-poses; *dynamic* classes (vehicles, two-wheelers, pedestrians) are taken from ± 1 frames only, since full accumulation smears moving objects into lane-long streaks. Free space is carved by ray-stepping the current sweep at 0.4 m intervals; coarser stepping (an early 1.7 m version) left most above-road voxels unknown, and the trained model hallucinated confident structure in exactly those unsupervised regions.

3.7 Agent futures and stationary flags

The same camera-confirmed instances used for 3D boxes are tracked through the annotation timeline to yield, per agent, six future offsets at 0.5 s spacing (3 s horizon). A binary stationary flag is derived on the fly during training as $|\text{displacement@3s}| < 0.5$ m; 32% of valid agents are stationary (parked rows), giving the flag a balanced signal. For vulnerable road users the camera-confirmation overlap

threshold is relaxed from 0.3 to 0.15—their small 2D projections are dominated by geometric error, and the strict gate silently deleted a third of them; relaxing it raised VRU recall from 0.08 to 0.25 in one round.

3.8 Unknown objects from occupancy blobs

Cones, guide posts and debris matter for driving but are absent from the annotation taxonomy—no label source exists. The occupancy GT already localises them: small blobs of the generic *obstacle* class ($\leq$2.4 m^2 footprint, $\leq$1.6 m tall) are extracted per frame and their centroids become centre-point GT for a dedicated heatmap head that joins the 3D-box stream as a third class with a fixed 0.4×0.4 m footprint. The factory thus *invents* supervision for a class no annotator ever drew.

Two revisions were required before this supervision became learnable. First, per-frame blobs are too sparse: the GT was rebuilt by accumulating LiDAR small-obstacle candidates over the *whole drive* in a global grid (confirmed after $\geq$3 hits), subtracting known-box footprints, and projecting the confirmed obstacles back into every frame as a dense occupancy mask. Second—and diagnostically decisive—a drive-accumulated mask marks obstacles the cameras *cannot see in that frame*: on urban validation scenes a BEV ray-march over the per-frame LiDAR raster (camera height 1.75 m against each cell's max-height profile) shows a median 59% of accumulated positives are occluded at any given moment. Training against them punishes physically impossible predictions and pins object-level recall near zero; relabelling occluded cells as per-cell *don't-care* makes both the loss and the metric refer only to the visible set. A complementary decode-time path lifts the per-camera 2D *obstacle* detections through the predicted depth into BEV (ground-contact sampling, cross-camera de-duplication), providing instance-separated unknown markers without any BEV-head involvement.

3.9 Ego-relevant traffic-light state

CoMET's TLR subnet already writes per-image light boxes with colour states into the 2D autolabels; we reduce them to one whole-image label per frame with an ego-relevance heuristic: the front telephoto camera wins outright (its narrow field of view looks straight down the travel corridor), else the front wide camera restricted to the central 50% of the image; the largest confirmed box supplies the state, and a 3-frame temporal median removes single-frame flickers. Classes are none/green/yellow/red.

3.10 Near-range risk field

A continuous potential field on a $\pm40\times\pm25$ m grid: each agent contributes an anisotropic Gaussian lobe that grows and leads with its GT speed (comet-shaped for vehicles, isotropic for VRUs, tight for parked vehicles), the static world contributes a distance falloff from occupied voxels, lobes combine saturatingly ($1-e^{-1.6\Sigma}$), and the field is weighted toward the ego neighbourhood. The definition composes upstream gates—camera-confirmed agents, ray-carved occupancy—so cleansing propagates.

3.11 Vector lane graph

The factory's vectorisation stage already produces connected polyline maps per scene; per frame we transform them into the ego frame, clip to the ROI ($x\in[-10,60]$ m, $|y|\leq25$ m), resample every chain to 12 points, keep the 24 nearest slots over {laneline, road edge, stopline}, and record an adjacency matrix linking consecutive pieces and near-touching endpoints ($<$1 m)—a planner-consumable structure rather than a raster.

3.12 Indoor / GNSS-dead exclusion

Underground garages defeat pose estimation *silently*: the fused pose drifts smoothly, so kinematic sanity checks pass while every pose-derived target (E2E, futures, temporal warps, BEV accumulation) is fiction. We detect the *environment* instead of the failure: the mean sky fraction of the front-wide 2D segmentation autolabel is $\approx$0.000 indoors versus 0.35+ outdoors even at night; thresholding at 0.02 flags 99 of 3,021 scenes (including three validation scenes that had been silently polluting E2E metrics), which are excluded from both training and validation.

3.13 Quality gates

Three train-time filters removed systematic label noise: scene-end trimming (the last 10 frames lack forward accumulation), a per-frame GT-coverage filter (labeled fraction $\geq$3% in the $\pm$30 m band and $\geq$0.5% in the 30–80 m band; long-stationary frames fail this), and indoor/underground scene rejection via panoptic sky statistics, because NDT [142] pose drift makes kinematic filters unreliable there.

4 METEOR Architecture

Fig. 1 shows the network. Eight 768×432 images pass through a shared ResNet-34 [2] with an FPN [8] that fuses all levels at stride 4 into a 160-channel feature f.

4.1 Depth-gated IPM

A fixed BEV grid (800×500 at 0.2 m; $\pm80\times\pm50$ m, z=0) is projected into every camera by the calibrated K, T. At each projection we `grid_sample` a 96-channel context feature c and the predicted depth distribution $p(d)$, then `gather` the probability at the point's geometric range r:

$$w_{i,n} = p_n(d{=}r_{i,n}) + \epsilon, \qquad b_i = \frac{\sum_n w_{i,n}\, c_n(u_{i,n})}{\sum_n w_{i,n}}, \quad (1)$$

where n indexes cameras, i BEV cells, and ϵ=0.05 keeps the gate leaky. Depth thus acts as a *visibility valve*: features flow into a BEV cell only from cameras whose predicted depth agrees with that cell's range, suppressing the occlusion bleed-through that plagues plain IPM. The block has

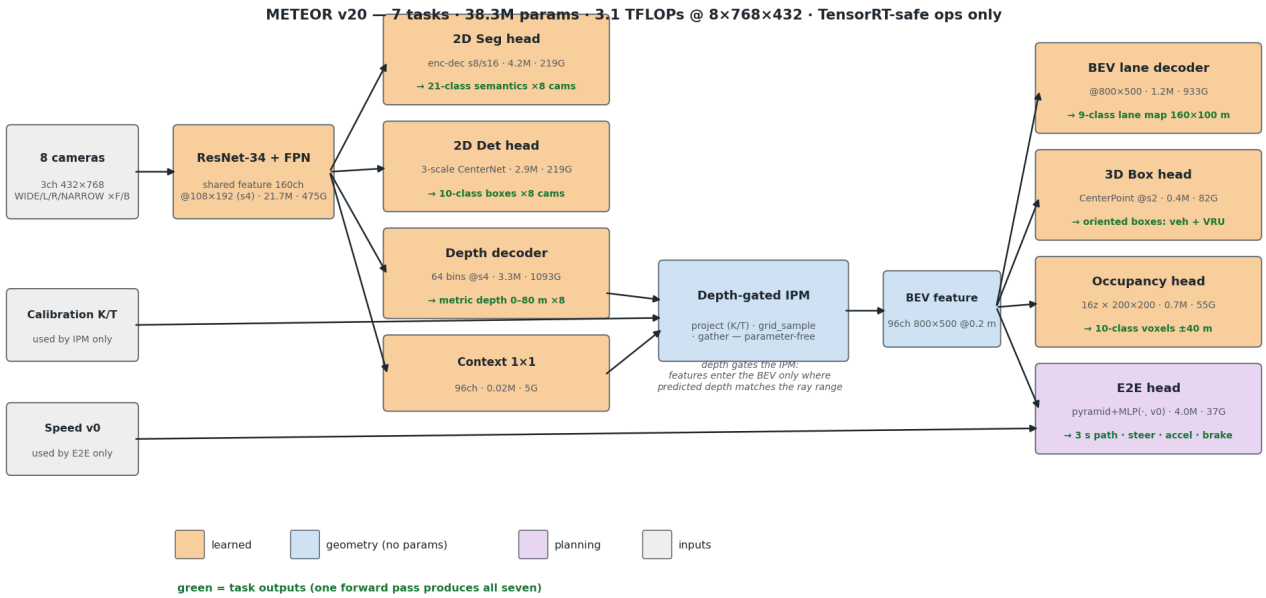

Figure 1: METEOR architecture. A shared ResNet-34+FPN yields a stride-4 feature per camera, consumed by four image-space heads. The parameter-free depth-gated IPM projects BEV grid points into the cameras and gathers context features weighted by the predicted depth probability at each point's range; four BEV-space heads (lanes, 3D boxes, occupancy, E2E) share the resulting 96-channel BEV feature. Green lines denote task outputs; all operators are TensorRT-exportable.

no parameters and uses only `grid_sample`/`gather`, in contrast to scatter-based splatting [49] or attention [57].

4.2 Heads and capacity allocation

Table 1 lists all stages. Two rules organise the design:

Parameters at low resolution, compute at high resolution. Early heads ran 3×3 convolutions at stride 4—the worst corner of the FLOPs/params trade-off. The redesigned 2D heads push channels into stride-8/16 towers (encoder–decoder for segmentation; a three-scale CenterNet pyramid for detection, with GT assigned by box size) and touch stride 4 only through 1×1 laterals: an 18× parameter increase for ∼2× FLOPs, worth +0.076 2D-seg mIoU (Sec. 6.5).

Tasks are cheap on a shared BEV. The occupancy head (a stride-2 stem on a ±40 m crop predicting 10×16 channels) and the E2E head (a strided pyramid pooled to 256-d, concatenated with speed v_0, and decoded by an MLP) together cost 3% of total compute.

4.3 Temporal memory and task routing

Since v22 the BEV is temporal: the previous frame's raw BEV, warped into the current frame by the relative ego pose (affine grid + sampling, no RNN), is fused residually; v29 generalises this to a three-slot memory at $t-0.4/1.2/2.8$ s with per-slot warps, covering occlusion persistence as well as velocity. The recurrence lives *outside* the graph: at deployment the history BEVs and warp matrices are ordinary engine inputs and the current raw BEV is an ordinary output, so the graph stays static. Crucially, tasks are *routed*: lanes, 3D boxes and occupancy read the raw single-frame BEV (temporal warps smear static geometry with moving-object ghosts), while the E2E head and agent forecasting read the fused BEV where velocity lives. Routing was verified numerically by perturbing the history inputs and checking output invariances. When a new fusion module replaces a trained one across rounds, its weights are *grafted*, not re-initialised: a zero-initialised fusion is function-preserving for the geometry heads but starves the motion heads of the transformed features they were trained on, and their heavily weighted losses then degrade the shared backbone—a failure we measured (BEV mIoU 0.283→0.211) before adopting grafting, after which the fused output is bit-equal to the previous round's at initialisation.

Table 1: Measured cost per stage (eight cameras, 768×432).

Stage	Params	GFLOPs	Share
Backbone + FPN	21.66 M	475	15%
Depth decoder	3.29 M	1093	35%
BEV lane decoder	1.16 M	933	30%
2D segmentation head	4.18 M	219	7%
2D detection head	2.85 M	219	7%
3D box head	0.41 M	82	3%
Occupancy head	0.70 M	55	2%
E2E head	4.04 M	37	1%
Context + IPM	0.02 M	6	<1%
Total	38.31 M	∼3118	

4.4 Multimodal trajectories

Single-trajectory imitation averages over genuinely ambiguous futures (straight vs. turn) and produces the geometric mean of both—a path through neither. The ego head therefore predicts $K=3$ trajectory hypotheses with mode confidences, trained winner-takes-all: only the hypothesis closest

to the GT receives the (curvature-weighted) regression loss, and a cross-entropy trains the confidences toward the winner. The same widening applies to per-agent forecasting. Modes warm-start as copies of the previously trained single mode with symmetry-breaking noise.

Pure winner-takes-all collapsed in practice: an initially slightly better mode won every frame, the losers received no gradient, and all three hypotheses converged to near-identical copies (one mode won 80/80 validation frames). Two changes restored diversity: ε-WTA (ε=0.1 of the loss always flows to the losing modes) and a directionally diverse initialisation (straight / left / right lateral biases of ± 0.8 m at 3 s). Because only 7.5% of moving frames turn more than 4 m laterally, turn frames are additionally oversampled $3\times$ by the per-epoch subset sampler.

4.5 Optional LiDAR input: one set of weights, two sensor configs

The network is camera-only, but LiDAR—already on the recording platform—can be accepted as an *optional* input without a second model. Points are projected into the cameras as a sparse metric depth map (the same format as the depth GT), and wherever a return exists the predicted depth softmax is blended toward the measured bin,

$$p' \;=\; (1-\alpha m)\,p \;+\; \alpha m\,\mathrm{tri}(d),$$

with $\mathrm{tri}(d)$ the two-bin triangular encoding of the measured depth, m the per-pixel return mask and α a learned scalar. All operations are elementwise, so the TensorRT graph is unchanged; feeding zeros makes $m \equiv 0$ and the network is *bit-identical* to the camera-only model, so a single checkpoint and a single engine serve both sensor configurations. Training hides the LiDAR input on half the samples (modality dropout), which keeps normalisation statistics calibrated for both modes and lets LiDAR-assisted gradients regularise the camera-only path. The depth distribution is exactly where this architecture localises its uncertainty (Sec. 4.1), so the measurement is injected at the single point of maximum effect for near-zero cost.

5 Training

5.1 Multi-task loss

The total loss is a weighted sum over tasks: BEV lanes (class-weighted cross-entropy with trained background, Lovász-softmax [138], boundary re-weighting, Tversky [139] on line classes, Dice [140] on crosswalks, and far-row weighting), depth (label-smoothed [136] bin cross-entropy plus an L1 penalty on expected metric depth), 3D boxes (Gaussian focal [27, 31] + centre L1; IoU-based box losses [141] were unnecessary at our grid resolution), 2D segmentation and detection (class- and camera-weighted variants), occupancy (class-weighted cross-entropy ignoring unknown voxels), and E2E (below). Aux losses return a graph-preserving zero on all-ignore batches—during rolling GT re-extraction whole batches can be unlabeled, and an unguarded mean cross-entropy returns NaN and destroys training within 50 steps.

5.2 Measurement-driven loss corrections

Every correction below was made *after* quantifying a failure, and we report them as results in Sec. 6: (i) **curvature weighting**: 83% of frames are near-straight and longitudinal waypoint magnitudes exceed lateral ones $18\times$, so plain L1 learns a go-straight prior; we weight lateral errors $4\times$ and scale each sample by $1 + \min(|y_{3s}|, 6)/1.5$. (ii) **coverage-based thin-class rasterisation** and (iii) **small-object retention** as described in Sec. 3.4, paired with per-class positive weights in both detection heads and a near-range (<20 m) positive boost for vulnerable road users in the 3D head. (iv) **Normalisation-statistics pollution**: with the 3-slot memory queue, each optimisation step runs the backbone three times on *history* frames for every current frame, and ∼16% of history slots are all-zero padding; BatchNorm running statistics were dominated by this off-distribution mixture and evaluation-mode accuracy collapsed within 50 steps while the training loss stayed healthy. Wrapping the history passes in `eval()` (they are inference, not supervision) fixed the collapse (step-50 probe mIoU $0.213 \to 0.265$, vehicle precision $0.48 \to 0.83$); a step-level metric probe every 500 steps (∼90 s) is what made a within-epoch failure visible at all. (v) **Oncoming-heading flips**: forecast headings for same-direction traffic have a 1–2.5% flip rate (>120° error at 3 s), but oncoming traffic flips 21–23% while the ego moves and **51%** while the ego is stopped—the median error stays low, so the failure is a bimodal flip toward the ego-forward majority prior, invisible in mean metrics. The forecast stem's only direction cue was the temporal smear in the fused BEV. We now feed an explicit *motion residual* (current BEV minus the ego-warped $t-0.4$ s slot, gated to zero when the slot is empty), under which an oncoming vehicle reads as a signed dipole along its true motion, and weight oncoming-vehicle cells $2.5\times$ in the forecast loss (the same minority treatment as VRUs). (vi) **Task starvation**: traffic-light accuracy regressed $0.88 \to 0.28$ when four new task losses were added in one round; raising its loss share restored it to 0.86 with no cost to the priority tasks.

5.3 Rolling rounds

Training proceeds in ∼24 h rounds on seven GPUs. Each round refreshes the scene list with newly converted recordings, warm-starts from the previous best checkpoint (shape-mismatched heads are dropped and re-initialised, so class-count changes retrain only the affected head), and applies whatever GT or loss fixes the previous round's metrics motivated. Nine rounds and four new task heads shipped in the project's first three days; by week three the loop had produced ∼40 rounds, with BEV mIoU rising 0.302→0.334 and curve ADE falling 0.72→0.39 m with no human intervention. Post-hoc residual refiners (zero-initialised U-Nets/MLPs on the frozen model's outputs, one per task) are trained between rounds and grafted back as trainable heads, so each round inherits the previous round's corrections. The same loop is

Table 2: Validation results on a held-out recording.

Metric	Value
BEV lane mIoU (8 classes + bg)	0.334
2D segmentation mIoU (21 classes)	0.555
3D det. vehicle precision / near-corridor recall	0.78 / 0.71
3D det. yaw axis error / direction flips	3.7° / 6%
3D det. VRU precision / near-corridor recall	0.75 / 0.46
E2E trajectory ADE / $\text{ADE}_{\text{curve}}$ / FDE (3 s)	0.87 m / 0.39 m / 1.89 m
Steering MAE / brake accuracy	0.032 rad / 0.96
Agent forecast ADE (3 s) / stationary-flag acc.	1.65 m / 0.70
Traffic-light state accuracy	0.84
Risk-field L1 (all / high-risk cells)	0.092 / 0.122
Occupancy-flow EPE moving / static	1.40 / 0.24 m/s

designed to run for months as the factory converts the full corpus.

6 Experiments

6.1 Setup

Data: 9,600+ scenes (≈80 driving hours, 1.25M keyframes) recorded nationwide across Japan by the DRS fleet-recording platform and converted progressively by the factory; new corpora are folded in round by round. Validation is one held-out recording session (273 scenes, different day and route) never used for training; a further 16-scene hold-out from the newest corpus serves as an untouched test set. Training: 8× data-parallel, batch 2 per GPU, AdamW [133, 134], cosine schedule [135], 4×10^{-5} peak LR on later rounds, 46k samples per epoch. All numbers below are single-model, single-pass; no test-time augmentation.

6.2 Main results

Table 2 summarises the current operating point. We stress the context: **every number is achieved without a single human label and without a single line of human-written code**, and the validation recording is from a different day and route than all training data; indoor (GNSS-dead) segments are excluded from both sides by the sky-fraction gate. The lane-graph head has not yet left its floor (precision/recall ≈0.01; its anchored-slot decoder appears structurally unable to model the relational task and a query-decoder rework is queued) and is reported qualitatively only. Fig. 2 shows qualitative output.

6.3 Zero-shot cross-country generalisation

The training corpus is Japan-only (left-hand traffic, Japanese vehicles and traffic lights). Running the unmodified TensorRT engine on US recordings from the same camera platform—right-hand traffic, US vehicle types, horizontally-mounted signals, and 114 km/h highway speeds absent from training—produces qualitatively correct multi-lane reconstruction, 3D detection of US trucks, and correct green-light state on the horizontal fixtures. We attribute this to the architectural split of Sec. 4.1: the image→BEV correspondence is computed from calibration rather than learned from data, so it cannot break when the country changes; only appearance must transfer. Observed failure modes are consistent with that reading: over-extended road-surface segmentation on unfamiliar pavement colours and confident traffic-light states on signal-free highways—appearance-level errors, not geometric ones.

6.4 Curvature weighting for imitation

Binning validation frames by ground-truth lateral displacement at 3 s exposed the go-straight prior of the unweighted model: ADE was 0.62 m on straight frames but 1.30 m at 2–4 m lateral and **2.29 m** beyond 4 m—3.7× worse than straight, hidden by an aggregate ADE of 0.86 m because 76% of validation frames are straight. One round of curvature-weighted training (Sec. 5.2) brought the curve subset to **1.07 m** and a second to **0.72 m**, at no cost to straight frames (aggregate ADE improved to 0.76 m). We consider the diagnosis-metric (curve-conditioned ADE, reported every epoch) as important as the fix: without it the regression would have been invisible.

6.5 Head capacity re-balancing

Replacing the stride-4 2D heads with the low-resolution-parameter design (Sec. 4) and fixing small-object GT retention raised 2D-seg mIoU from 0.375 to **0.451** in one round (lane 0.402→0.483, pole 0.244→0.449) and enabled detection of classes that had never fired before (traffic lights, distant signs, road obstacles), at +10% total FLOPs and +37% parameters.

6.6 Occupancy supervision density

With 1.7 m ray steps, most above-road voxels remained *unknown* and the model hallucinated high-confidence structure there (a solid "slab" over the road in 3D renderings), while measured road IoU was unaffected—the errors live exactly where no supervision exists. Densifying carving to one sample per 0.4 m voxel multiplies free-space supervision and is expected to remove the hallucination; per-range road IoU (Table 2) already shows no near-field deficit, countering the visual impression given by top-down renderings that flatten hallucinated upper voxels onto the road.

7 Deployment

The entire network exports as one static ONNX graph (all task outputs plus the raw BEV for the streaming recurrence); `affine_grid` is replaced at export by a constant base grid and a matrix product, after which every operator is TensorRT-native—no plugins. ONNX-Runtime outputs match PyTorch to $\sim10^{-4}$ on all heads. A stock fp16 build on TensorRT 8.6 runs the current eight-camera, all-task graph (including the grafted refiners and the dense unknown head) at 9.3 inferences/s (108 ms/frame, p99 117 ms) on a single L40S. The temporal memory is a device-resident ring buffer maintained

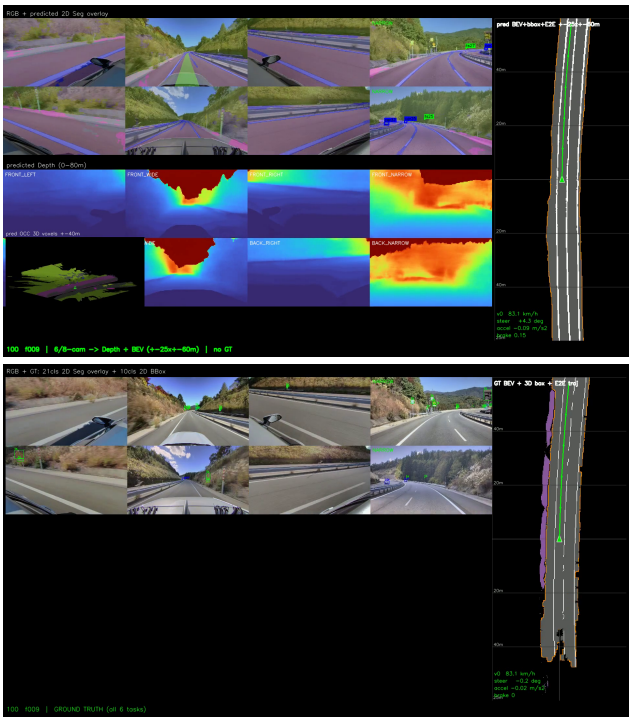

Figure 2: Qualitative results. Top: single-pass inference—eight cameras with 21-class segmentation, 2D and projected 3D boxes, and the predicted path ribbon; metric depth; predicted occupancy (isometric); BEV lanes with oriented boxes and the E2E trajectory with control gauges. Bottom: the corresponding fully automatic ground truth.

by the host runtime: history BEVs are spliced by device-to-device copies (the naïve host-staged variant moved 460 MB per frame and cost 3.9× the latency), the raw BEV returns as an output, and the graph itself remains feed-forward.

fp16 deployment exposed two overflow modes that PyTorch's autocast masks: $\log(1+e^x)$ in the decoupled-planner speed decoder overflows at highway speed logits, and a global average pool over 10^5 BEV cells overflows an fp16 accumulator once temporal fusion is active. TensorRT 8.6 cannot pin per-layer precision inside Myelin-fused regions, so both were fixed *by construction*: the numerically stable softplus $\max(x,0)+\log 1p(e^{-|x|})$ and a scale-before-sum pooling whose partial sums are bounded by $\max|x|$. A per-layer engine profile attributes 51% of the latency to the BEV projection/encoder block and ∼1.5% to all sparse task heads combined—removing tasks barely helps, while camera count, INT8 and an asymmetric BEV grid (80 m ahead / 40 m behind) are the effective levers for embedded targets.

8 Agent-Authored Development

Every artefact of this project—the ten GT extractors, twenty model variants, the trainer, the demo renderers, the documentation, and this manuscript—was written by a large language model agent (Claude Fable 5, Anthropic) operating in an autonomous loop: humans supplied goals and critiques in natural language (≈70 instructions over the reported period) and reviewed rendered videos and metrics; the agent designed, implemented, launched and monitored training, diagnosed failures from its own measurements, and iterated. Several of the fixes reported above originated from the agent's own diagnostics (e.g., the KMAX truncation analysis); others from human observations that the agent then localised and repaired (e.g., the scene-tail speed bug). We do not claim the agent is autonomous in the strong sense—direction remained human—but the division of labour (humans specify *what*, the agent implements, measures, and explains *how*) held for 100% of the code, and we found the resulting cadence (nine training rounds and four new task heads in three days) difficult to reconcile with conventional development.

9 Limitations

Thin-class BEV IoU (lane line 0.14, stop line 0.11) remains low in absolute terms; the labels themselves carry residual noise near intersections, and IoU on 1-px-wide classes is a harsh metric. The E2E head is pure imitation without commands or multi-modality, so intersections are resolved by data prior rather than intent. Occupancy hallucination in unknown regions is mitigated but not eliminated by denser carving; a principled treatment of camera visibility masks [85] is future work. Validation uses a single held-out recording; broader-scale evaluation across platforms awaits the full corpus conversion. Finally, the network is single-frame; temporal fusion is an obvious next step.

10 Conclusion

METEOR demonstrates that a competitive seven-task surround-view driving network can be built from raw fleet recordings with no human annotation and, in this instance, no human-written code—the autolabel factory of the Co-MLOps project (CoMET) supplies every supervision signal, and an LLM agent supplied every line of implementation. We believe the combination—data platforms that record everything, factories that label everything, agents that implement anything specified—changes the economics of adding a task from "a labeling campaign" to "a prompt and a GPU-day," and we offer the system, its measurements, and its failure catalogue as evidence.

Acknowledgements. Built on the CoMET autolabeling foundation of the Co-MLOps project. Presented at NVIDIA GTC 2026 (session S81897).

References

[1] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.

[2] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

[3] M. Tan and Q. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *ICML*, 2019.

[4] Z. Liu et al. A ConvNet for the 2020s. In *CVPR*, 2022.

[5] A. Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

[6] Z. Liu et al. Swin Transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021.

[7] J. Wang et al. Deep high-resolution representation learning for visual recognition. *IEEE TPAMI*, 2020.

[8] T.-Y. Lin et al. Feature pyramid networks for object detection. In *CVPR*, 2017.

[9] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.

[10] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *CVPR*, 2015.

[11] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *MICCAI*, 2015.

[12] H. Zhao et al. Pyramid scene parsing network. In *CVPR*, 2017.

[13] L.-C. Chen et al. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *ECCV*, 2018.

[14] E. Xie et al. SegFormer: Simple and efficient design for semantic segmentation with transformers. In *NeurIPS*, 2021.

[15] C. Yu et al. BiSeNet: Bilateral segmentation network for real-time semantic segmentation. In *ECCV*, 2018.

[16] E. Romera et al. ERFNet: Efficient residual factorized ConvNet for real-time semantic segmentation. *IEEE T-ITS*, 2018.

[17] A. Kirillov et al. Panoptic segmentation. In *CVPR*, 2019.

[18] B. Cheng et al. Panoptic-DeepLab: A simple, strong, and fast baseline for bottom-up panoptic segmentation. In *CVPR*, 2020.

[19] T.-Y. Lin et al. Microsoft COCO: Common objects in context. In *ECCV*, 2014.

[20] M. Cordts et al. The Cityscapes dataset for semantic urban scene understanding. In *CVPR*, 2016.

[21] G. Neuhold et al. The Mapillary Vistas dataset for semantic understanding of street scenes. In *ICCV*, 2017.

[22] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *NeurIPS*, 2015.

[23] K. He, G. Gkioxari, P. Dollár, and R. Girshick. Mask R-CNN. In *ICCV*, 2017.

[24] W. Liu et al. SSD: Single shot multibox detector. In *ECCV*, 2016.

[25] J. Redmon et al. You only look once: Unified, real-time object detection. In *CVPR*, 2016.

[26] J. Redmon and A. Farhadi. YOLOv3: An incremental improvement. *arXiv:1804.02767*, 2018.

[27] T.-Y. Lin et al. Focal loss for dense object detection. In *ICCV*, 2017.

[28] Z. Tian et al. FCOS: Fully convolutional one-stage object detection. In *ICCV*, 2019.

[29] S. Zhang et al. Bridging the gap between anchor-based and anchor-free detection via adaptive training sample selection. In *CVPR*, 2020.

[30] H. Law and J. Deng. CornerNet: Detecting objects as paired keypoints. In *ECCV*, 2018.

[31] X. Zhou, D. Wang, and P. Krähenbühl. Objects as points. *arXiv:1904.07850*, 2019.

[32] N. Carion et al. End-to-end object detection with transformers. In *ECCV*, 2020.

[33] X. Zhu et al. Deformable DETR: Deformable transformers for end-to-end object detection. In *ICLR*, 2021.

[34] X. Pan et al. Spatial as deep: Spatial CNN for traffic scene understanding. In *AAAI*, 2018.

[35] Z. Qin, H. Wang, and X. Li. Ultra fast structure-aware deep lane detection. In *ECCV*, 2020.

[36] L. Tabelini et al. Keep your eyes on the lane: Real-time attention-guided lane detection. In *CVPR*, 2021.

[37] L. Liu et al. CondLaneNet: A top-to-down lane detection framework based on conditional convolution. In *ICCV*, 2021.

[38] T. Zheng et al. CLRNet: Cross layer refinement network for lane detection. In *CVPR*, 2022.

[39] L. Tabelini et al. PolyLaneNet: Lane estimation via deep polynomial regression. In *ICPR*, 2020.

[40] N. Garnett et al. 3D-LaneNet: End-to-end 3d multiple lane detection. In *ICCV*, 2019.

[41] Y. Guo et al. Gen-LaneNet: A generalized and scalable approach for 3d lane detection. In *ECCV*, 2020.

[42] L. Chen et al. PersFormer: 3d lane detection via perspective transformer and the OpenLane benchmark. In *ECCV*, 2022.

[43] Q. Li et al. HDMapNet: An online hd map construction and evaluation framework. In *ICRA*, 2022.

[44] Y. Liu et al. VectorMapNet: End-to-end vectorized hd map learning. In *ICML*, 2023.

[45] B. Liao et al. MapTR: Structured modeling and learning for online vectorized hd map construction. In *ICLR*, 2023.

[46] B. Liao et al. MapTRv2: An end-to-end framework for online vectorized hd map construction. *arXiv:2308.05736*, 2023.

[47] W. Ding et al. PivotNet: Vectorized pivot learning for end-to-end hd map construction. In *ICCV*, 2023.

[48] T. Yuan et al. StreamMapNet: Streaming mapping network for vectorized online hd map construction. In *WACV*, 2024.

[49] J. Philion and S. Fidler. Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d. In *ECCV*, 2020.

[50] J. Huang et al. BEVDet: High-performance multi-camera 3d object detection in bird-eye-view. *arXiv:2112.11790*, 2021.

[51] J. Huang and G. Huang. BEVDet4D: Exploit temporal cues in multi-camera 3d object detection. *arXiv:2203.17054*, 2022.

[52] Y. Li et al. BEVDepth: Acquisition of reliable depth for multi-view 3d object detection. In *AAAI*, 2023.

[53] J. Park et al. Time will tell: New outlooks and a baseline for temporal multi-view 3d object detection. In *ICLR*, 2023.

[54] Y. Wang et al. DETR3D: 3d object detection from multi-view images via 3d-to-2d queries. In *CoRL*, 2021.

[55] Y. Liu et al. PETR: Position embedding transformation for multi-view 3d object detection. In *ECCV*, 2022.

[56] Y. Liu et al. PETRv2: A unified framework for 3d perception from multi-camera images. In *ICCV*, 2023.

[57] Z. Li et al. BEVFormer: Learning bird's-eye-view representation from multi-camera images via spatiotemporal transformers. In *ECCV*, 2022.

[58] X. Lin et al. Sparse4D: Multi-view 3d object detection with sparse spatial-temporal fusion. *arXiv:2211.10581*, 2022.

[59] S. Wang et al. Exploring object-centric temporal modeling for efficient multi-view 3d object detection. In *ICCV*, 2023.

[60] B. Zhou and P. Krähenbühl. Cross-view transformers for real-time map-view semantic segmentation. In *CVPR*, 2022.

[61] A. W. Harley et al. Simple-BEV: What really matters for multi-sensor bev perception? In *ICRA*, 2023.

[62] E. Xie et al. M^2BEV: Multi-camera joint 3d detection and segmentation with unified bird's-eye view representation. *arXiv:2204.05088*, 2022.

[63] Y. Li et al. Fast-BEV: A fast and strong bird's-eye view perception baseline. *arXiv:2301.12511*, 2023.

[64] T. Wang et al. FCOS3D: Fully convolutional one-stage monocular 3d object detection. In *ICCV Workshops*, 2021.

[65] C. Reading et al. Categorical depth distribution network for monocular 3d object detection. In *CVPR*, 2021.

[66] H. A. Mallot et al. Inverse perspective mapping simplifies optical flow computation and obstacle detection. *Biological Cybernetics*, 1991.

[67] M. Bertozzi and A. Broggi. GOLD: A parallel real-time stereo vision system for generic obstacle and lane detection. *IEEE Trans. Image Processing*, 1998.

[68] J. Kannala and S. S. Brandt. A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses. *IEEE TPAMI*, 2006.

[69] Y. Zhou and O. Tuzel. VoxelNet: End-to-end learning for point cloud based 3d object detection. In *CVPR*, 2018.

[70] Y. Yan, Y. Mao, and B. Li. SECOND: Sparsely embedded convolutional detection. *Sensors*, 2018.

[71] A. H. Lang et al. PointPillars: Fast encoders for object detection from point clouds. In *CVPR*, 2019.

[72] S. Shi, X. Wang, and H. Li. PointRCNN: 3d object proposal generation and detection from point cloud. In *CVPR*, 2019.

[73] S. Shi et al. PV-RCNN: Point-voxel feature set abstraction for 3d object detection. In *CVPR*, 2020.

[74] T. Yin, X. Zhou, and P. Krähenbühl. Center-based 3d object detection and tracking. In *CVPR*, 2021.

[75] X. Bai et al. TransFusion: Robust lidar-camera fusion for 3d object detection with transformers. In *CVPR*, 2022.

[76] Z. Liu et al. BEVFusion: Multi-task multi-sensor fusion with unified bird's-eye view representation. In *ICRA*, 2023.

[77] S. Song et al. Semantic scene completion from a single depth image. In *CVPR*, 2017.

[78] J. Behley et al. SemanticKITTI: A dataset for semantic scene understanding of lidar sequences. In *ICCV*, 2019.

[79] A.-Q. Cao and R. de Charette. MonoScene: Monocular 3d semantic scene completion. In *CVPR*, 2022.

[80] Y. Huang et al. Tri-perspective view for vision-based 3d semantic occupancy prediction. In *CVPR*, 2023.

[81] Y. Zhang, Z. Zhu, and D. Du. OccFormer: Dual-path transformer for vision-based 3d semantic occupancy prediction. In *ICCV*, 2023.

[82] Y. Wei et al. SurroundOcc: Multi-camera 3d occupancy prediction for autonomous driving. In *ICCV*, 2023.

[83] Y. Li et al. VoxFormer: Sparse voxel transformer for camera-based 3d semantic scene completion. In *CVPR*, 2023.

[84] X. Wang et al. OpenOccupancy: A large scale benchmark for surrounding semantic occupancy perception. In *ICCV*, 2023.

[85] X. Tian et al. Occ3D: A large-scale 3d occupancy prediction benchmark for autonomous driving. In *NeurIPS*, 2023.

[86] Z. Li et al. FB-OCC: 3d occupancy prediction based on forward-backward view transformation. *arXiv:2307.01492*, 2023.

[87] D. Eigen, C. Puhrsch, and R. Fergus. Depth map prediction from a single image using a multi-scale deep network. In *NeurIPS*, 2014.

[88] H. Fu et al. Deep ordinal regression network for monocular depth estimation. In *CVPR*, 2018.

[89] J. H. Lee et al. From big to small: Multi-scale local planar guidance for monocular depth estimation. *arXiv:1907.10326*, 2019.

[90] S. F. Bhat, I. Alhashim, and P. Wonka. AdaBins: Depth estimation using adaptive bins. In *CVPR*, 2021.

[91] C. Godard et al. Digging into self-supervised monocular depth estimation. In *ICCV*, 2019.

[92] R. Ranftl et al. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. *IEEE TPAMI*, 2022.

[93] L. Yang et al. Depth Anything: Unleashing the power of large-scale unlabeled data. In *CVPR*, 2024.

[94] J. Ku, A. Harakeh, and S. L. Waslander. In defense of classical image processing: Fast depth completion on the CPU. In *CRV*, 2018.

[95] F. Ma and S. Karaman. Sparse-to-dense: Depth prediction from sparse depth samples and a single image. In *ICRA*, 2018.

[96] D. A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In *NeurIPS*, 1988.

[97] M. Bojarski et al. End to end learning for self-driving cars. *arXiv:1604.07316*, 2016.

[98] F. Codevilla et al. End-to-end driving via conditional imitation learning. In *ICRA*, 2018.

[99] F. Codevilla et al. Exploring the limitations of behavior cloning for autonomous driving. In *ICCV*, 2019.

[100] M. Bansal, A. Krizhevsky, and A. Ogale. ChauffeurNet: Learning to drive by imitating the best and synthesizing the worst. In *RSS*, 2019.

[101] W. Zeng et al. End-to-end interpretable neural motion planner. In *CVPR*, 2019.

[102] A. Dosovitskiy et al. CARLA: An open urban driving simulator. In *CoRL*, 2017.

[103] D. Chen et al. Learning by cheating. In *CoRL*, 2020.

[104] A. Prakash, K. Chitta, and A. Geiger. Multi-modal fusion transformer for end-to-end autonomous driving. In *CVPR*, 2021.

[105] H. Shao et al. Safety-enhanced autonomous driving using interpretable sensor fusion transformer. In *CoRL*, 2022.

[106] P. Wu et al. Trajectory-guided control prediction for end-to-end autonomous driving: A simple yet strong baseline. In *NeurIPS*, 2022.

[107] A. Hu et al. Model-based imitation learning for urban driving. In *NeurIPS*, 2022.

[108] S. Hu et al. ST-P3: End-to-end vision-based autonomous driving via spatial-temporal feature learning. In *ECCV*, 2022.

[109] Y. Hu et al. Planning-oriented autonomous driving. In *CVPR*, 2023.

[110] B. Jiang et al. VAD: Vectorized scene representation for efficient autonomous driving. In *ICCV*, 2023.

[111] A. Geiger, P. Lenz, and R. Urtasun. Are we ready for autonomous driving? The KITTI vision benchmark suite. In *CVPR*, 2012.

[112] H. Caesar et al. nuScenes: A multimodal dataset for autonomous driving. In *CVPR*, 2020.

[113] H. Caesar et al. nuPlan: A closed-loop ml-based planning benchmark for autonomous vehicles. *arXiv:2106.11810*, 2021.

[114] P. Sun et al. Scalability in perception for autonomous driving: Waymo open dataset. In *CVPR*, 2020.

[115] B. Wilson et al. Argoverse 2: Next generation datasets for self-driving perception and forecasting. In *NeurIPS Datasets and Benchmarks*, 2021.

[116] F. Yu et al. BDD100K: A diverse driving dataset for heterogeneous multitask learning. In *CVPR*, 2020.

[117] J. Mao et al. One million scenes for autonomous driving: ONCE dataset. In *NeurIPS Datasets and Benchmarks*, 2021.

[118] C. R. Qi et al. Offboard 3d object detection from point cloud sequences. In *CVPR*, 2021.

[119] B. Yang et al. Auto4D: Learning to label 4d objects from sequential point clouds. *arXiv:2101.06586*, 2021.

[120] D.-H. Lee. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In *ICML Workshops*, 2013.

[121] A. Tarvainen and H. Valpola. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. In *NeurIPS*, 2017.

[122] K. Sohn et al. FixMatch: Simplifying semi-supervised learning with consistency and confidence. In *NeurIPS*, 2020.

[123] Q. Xie et al. Self-training with noisy student improves ImageNet classification. In *CVPR*, 2020.

[124] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.

[125] K. He et al. Masked autoencoders are scalable vision learners. In *CVPR*, 2022.

[126] M. Oquab et al. DINOv2: Learning robust visual features without supervision. *arXiv:2304.07193*, 2023.

[127] R. Caruana. Multitask learning. *Machine Learning*, 28(1), 1997.

[128] I. Kokkinos. UberNet: Training a universal convolutional neural network for low-, mid-, and high-level vision using diverse datasets and limited memory. In *CVPR*, 2017.

[129] A. R. Zamir et al. Taskonomy: Disentangling task transfer learning. In *CVPR*, 2018.

[130] A. Kendall, Y. Gal, and R. Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *CVPR*, 2018.

[131] Z. Chen et al. GradNorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *ICML*, 2018.

[132] T. Yu et al. Gradient surgery for multi-task learning. In *NeurIPS*, 2020.

[133] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

[134] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[135] I. Loshchilov and F. Hutter. SGDR: Stochastic gradient descent with warm restarts. In *ICLR*, 2017.

[136] C. Szegedy et al. Rethinking the Inception architecture for computer vision. In *CVPR*, 2016.

[137] P. Micikevicius et al. Mixed precision training. In *ICLR*, 2018.

[138] M. Berman, A. Rannen Triki, and M. B. Blaschko. The Lovász-softmax loss: A tractable surrogate for the optimization of the intersection-over-union measure in neural networks. In *CVPR*, 2018.

[139] S. S. M. Salehi, D. Erdogmus, and A. Gholipour. Tversky loss function for image segmentation using 3d fully convolutional deep networks. In *MLMI*, 2017.

[140] F. Milletari, N. Navab, and S.-A. Ahmadi. V-Net: Fully convolutional neural networks for volumetric medical image segmentation. In *3DV*, 2016.

[141] H. Rezatofighi et al. Generalized intersection over union: A metric and a loss for bounding box regression. In *CVPR*, 2019.

[142] P. Biber and W. Straßer. The normal distributions transform: A new approach to laser scan matching. In *IROS*, 2003.

[143] D. H. Douglas and T. K. Peucker. Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. *Cartographica*, 1973.

[144] T. Y. Zhang and C. Y. Suen. A fast parallel algorithm for thinning digital patterns. *Communications of the ACM*, 1984.

[145] M. Chen et al. Evaluating large language models trained on code. *arXiv:2107.03374*, 2021.

[146] Y. Li et al. Competition-level code generation with AlphaCode. *Science*, 378(6624), 2022.

[147] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

[148] C. E. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

[149] OpenAI. GPT-4 technical report. *arXiv:2303.08774*, 2023.